

分数：_____



深圳技术大学实践课程报告

课程名称：_____单片机原理与接口技术_____

课程编号：_____SG00407_____

项目名称：_____基于 STM32 的智能巡检蓝牙避障小车_____

任课教师：_____王蕊，陈赓亮_____

学 生：_____张欣悦_____学 号：_____202300703056_____

班 级：_____23 电科 2 班_____

报告/实践地点：_____D3-419_____

报告/实践时间：_____2025.06.06-2025.06.19_____

提交时间：_____2025 年 6 月 20 日星期四_____

1 原理与设计

本次单片机课程设计实验，我设计的是基于 STM32C8T6 芯片的智能避障蓝牙巡检的小车，该课程设计参考了江协科技。

避障控制 超声波测距：利用 HC-SR04 超声波发射超声波，计算回波时间差， $\text{距离} = (340\text{m/s} \times \Delta t) / 2$ 。同时在车头的超声模块在检测到距离障碍物 $< 15\text{cm}$ 时，装载超声模块的云台会通过舵机先向右旋转 90° 探测右边情况，如果右边有障碍物还是小于 15cm 时此时向左转 180° ，若是左边障碍物还是小于 15cm 的话，小车就会自动往后退，直到识别不到障碍物后继续往前走。向左转向右转都识别不到障碍物的话，小车就会自动向左转或向右转，继续前进。

蓝牙遥控 HC-06 模块：通过串口（UART）与 STM32 通信。手机 APP 发送指令，前进后退左转右转加速等，STM32 解析并执行动作。选择 UART 因协议简单、实时性强。

电机驱动 PWM 调速：STM32 定时器生成 PWM 波控制电机转速。TB6612FNG 芯片控制电流方向，IN1/IN2 电平组合决定正反转，注意电机接线问题！一侧电机的红线通过鳄鱼夹加在一起，白线也通过另外一个鳄鱼夹夹在一起，加上另一侧的两条线，总共 4 条线，再接到电机驱动模块上，这样一个电机模块就可以控制 4 个轮子

巡检模式 通过在车头装上 4 个红外循迹模块，左边两个，右边两个形成简易的巡检功能。小车的 4 个红外循迹模块在未识别到地上的黑线时引脚的值为 0，且红灯不亮，小车将通过程序设计依旧走直线；当模块被遮挡时引脚为 1，红灯亮，同时小车根据程序编写会循着黑线走。

主要硬件选型说明

主控（STM32F103C8T6）

避障传感器（HC-SR04 超声波）

电机驱动（TB6612FNG）

蓝牙模块（HC-06）

电源（18650 锂电池组）

直流电机 4 个

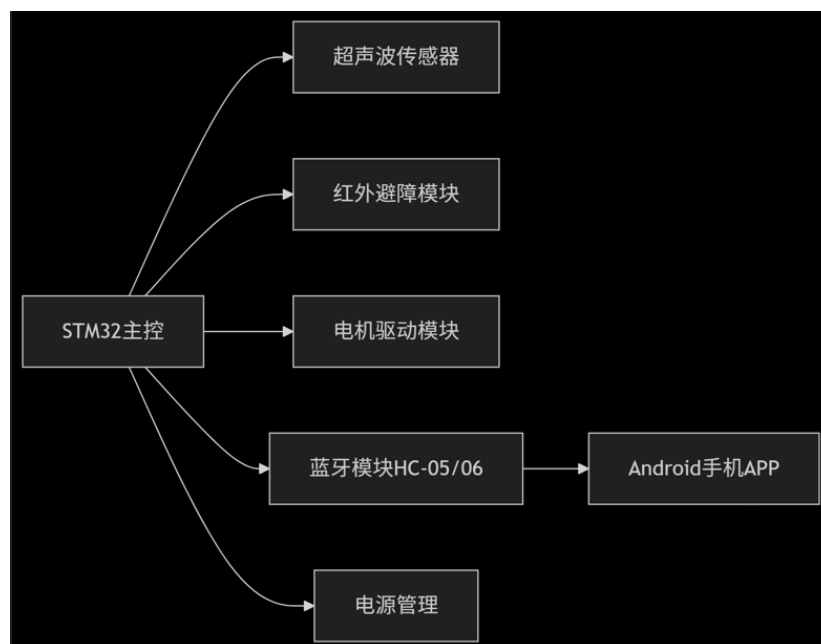
AMS1117 3.3V 5V 降压模块

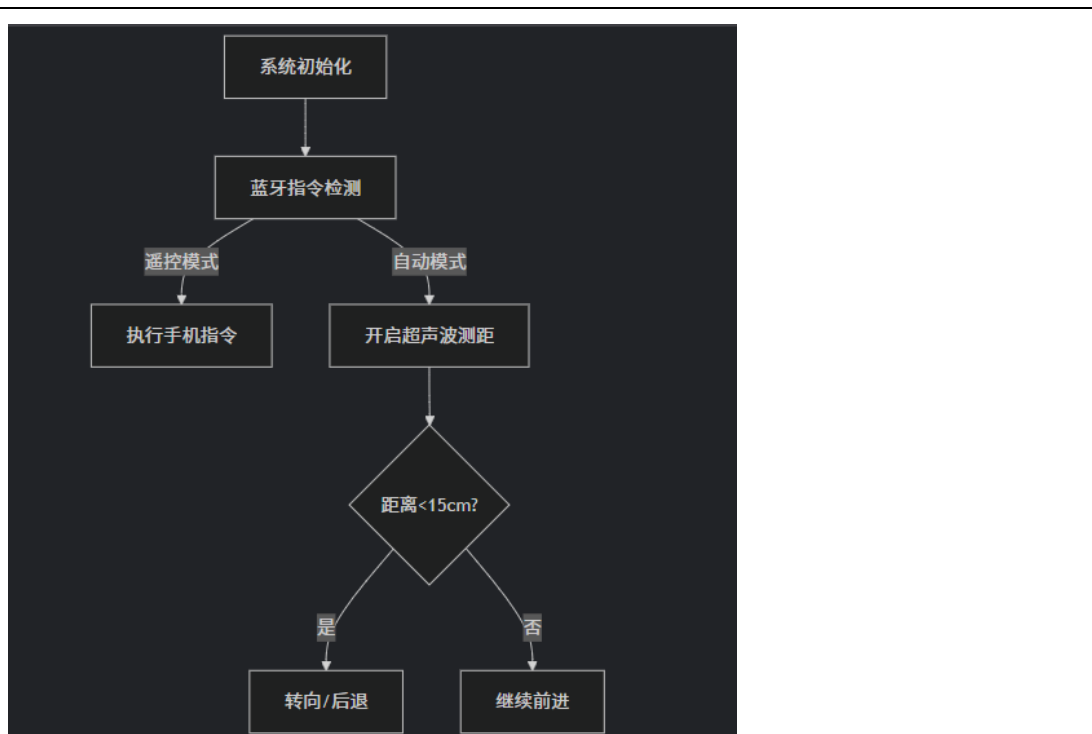
面包线 杜邦线若干

智能小车地盘

红外循迹传感器

2 核心框架





3 代码实现

(1) PWM 模块

设置两个 PWM 波, TIM2 时基单元是给四个电机的, TIM3 给舵机模块, 启动时钟源, 启动 GPIO 口, 接着初始化实际单元, 初始化输出比较单元使能 TIM。电机的 PWM 波 ARR 值设置为 100, PSC 设置为 36; 舵机的 Air 设置为 20000, PSC, 设置为 72

```

#include "stm32f10x.h" // Device header

void PWM_Init(void)
{
    //1. 启动时钟源
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE); //给4个电机的PWM, TIM2有两个通道, GPIOA口, 通道1
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3, ENABLE); //给舵机模块的PWM, GPIOB口, TIM3第4通道
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);
    //2. 启动GPIO口
    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_1|GPIO_Pin_2; //GPIOA1是通道2
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_1;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz; //初始化舵机的B1口
    GPIO_Init(GPIOB, &GPIO_InitStructure);
    //3. 初始化时基单元
    //TIM2是给四个电机的
    TIM_InternalClockConfig(TIM2);
    TIM_InternalClockConfig(TIM3);

    TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure; //这种写的后面再配一个PWM波就不用写了
    TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1;
    TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseInitStructure.TIM_Period = 100 - 1; //ARR
    TIM_TimeBaseInitStructure.TIM_Prescaler = 36 - 1; //PSC
    TIM_TimeBaseInitStructure.TIM_RepetitionCounter = 0;
    TIM_TimeBaseInit(TIM2, &TIM_TimeBaseInitStructure);

    TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1;
    TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseInitStructure.TIM_Period = 20000 - 1; //ARR
    TIM_TimeBaseInitStructure.TIM_Prescaler = 72 - 1; //PSC //因为舵机的周期是20ms, 1000ms是1s
    TIM_TimeBaseInitStructure.TIM_RepetitionCounter = 0; //f=50=72MHz/(ARR+1)/(PSC+1), 72000000
    TIM_TimeBaseInit(TIM3, &TIM_TimeBaseInitStructure);
    //4. 初始化输出比较单元
    TIM_OCInitTypeDef TIM_OCInitStructure; //这种写的后面再配一个PWM波就不用写了
    TIM_OCStructInit(&TIM_OCInitStructure);
    TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1;
    TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High;
    TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
    TIM_OCInitStructure.TIM_Pulse = 0; //CCR
    TIM_OC3Init(TIM2, &TIM_OCInitStructure);
    TIM_OC2Init(TIM2, &TIM_OCInitStructure);
    TIM_OC4Init(TIM3, &TIM_OCInitStructure);
    //5. 使能TIM
    TIM_Cmd(TIM2, ENABLE);
    TIM_Cmd(TIM3, ENABLE);

    //TIM3给舵机模块的
}

void PWM_SetCompare2(uint16_t Compare)
{
}

```

(2) 电机模块

```

#include "stm32f10x.h"           // Device header
#include "PWM.h"

void Motor_Init(void)
{
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_4 | GPIO_Pin_5 | GPIO_Pin_6 | GPIO_Pin_7;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    PWM_Init();
}

void Motor_SetLeftSpeed(int8_t Speed)
{
    if(Speed>0)
    {
        GPIO_SetBits(GPIOA, GPIO_Pin_4);
        GPIO_ResetBits(GPIOA, GPIO_Pin_5);
        PWM_SetCompare2(Speed);
    }
    else if(Speed==0)
    {
        GPIO_SetBits(GPIOA, GPIO_Pin_4);
        GPIO_SetBits(GPIOA, GPIO_Pin_5);
        PWM_SetCompare2(Speed);
    }
    else{
        GPIO_ResetBits(GPIOA, GPIO_Pin_4);
        GPIO_SetBits(GPIOA, GPIO_Pin_5);
        PWM_SetCompare2(-Speed);
    }
}

5 void Motor_SetRightSpeed(int8_t Speed)
6 {
7     if(Speed>0)
8     {
9         GPIO_SetBits(GPIOA, GPIO_Pin_6);
10        GPIO_ResetBits(GPIOA, GPIO_Pin_7);
11        PWM_SetCompare2(Speed);
12    }
13    else if(Speed==0)
14    {
15        GPIO_SetBits(GPIOA, GPIO_Pin_6);
16        GPIO_SetBits(GPIOA, GPIO_Pin_7);
17        PWM_SetCompare2(Speed);
18    }
19    else{
20        GPIO_ResetBits(GPIOA, GPIO_Pin_6);
21        GPIO_SetBits(GPIOA, GPIO_Pin_7);
22        PWM_SetCompare2(-Speed);
23    }
24 }

25 }
26 void PWM_SetCompare2(uint16_t Compare)
27 {
28     TIM_SetCompare2(TIM2, Compare);
29 }
30 void PWM_SetCompare3(uint16_t Compare)
31 {
32     TIM_SetCompare3(TIM2, Compare);    //自设定3个CCR的值，以调占空比，compare就是CCR的值
33 }
34 void PWM_SetCompare4(uint16_t Compare)
35 {
36     TIM_SetCompare4(TIM3, Compare);
37 }
38 }

```

(3) 小车控制模块

这一部分主要通过设置四个电机引脚的高低电平来调控小车，实现向左向右前进后退等功能

```
1
5 void CAR_Init(void)
3 {
7     Motor_Init();
3 }
3 void Go_Ahead() {
1     Motor_SetLeftSpeed(-70);
1     Motor_SetRightSpeed(-70);
2 }
3 void Go_Back() {
1     Motor_SetLeftSpeed(70);
5     Motor_SetRightSpeed(70);
3 }
7 void Turn_Left() {
3     Motor_SetLeftSpeed(0);
9     Motor_SetRightSpeed(-70);
1 }
1 void Turn_Right() {
2     Motor_SetLeftSpeed(-70);
3     Motor_SetRightSpeed(0);
1 }
5 void Self_Right() {
5     Motor_SetLeftSpeed(-70);
7     Motor_SetRightSpeed(70);
3 }
3 void Self_Left() {
1     Motor_SetLeftSpeed(70);
1     Motor_SetRightSpeed(-70);
2 }
3 void Car_Stop() {
1     Motor_SetLeftSpeed(0);
5     Motor_SetRightSpeed(0);
3 }
```

(4) 舵机模块

20ms, 其中高电平的宽度为 0.5ms 到 2.5ms, 0.5ms 的高电平除于 20ms 就是占空比, 即对应 CCR/ARR, 500/20000

同理, 2.5ms 的高电平除于 20ms 就是占空比, 即对应 CCR/ARR, 2500/20000

0 到 180 度对应 500 到 2500, 所以有公式

$\text{PWM_SetCompare4}(\text{Angle}/180*2000+500)$

```

1 #include "stm32f10x.h" // Device header
2 #include "PWM.h"
3 void Servo_Init(void)
4 {
5     PWM_Init();
6 }
7 void Servo_SetAngle(float Angle)
8 {
9     PWM_SetCompare4(Angle/180*2000+500);
10 }
1 //20ms, 其中高电平的宽度为0.5ms到2.5ms, 0.5ms的高电平除于20ms就是占空比
2 //同理, 2.5ms的高电平除于20ms就是占空比, 即对应CCR/ARR, 2500/20000
3 //0到180度对应500到2500, 所以有公式

```

(5) 蓝牙模块

```

uint8_t Serial_RxFlag;

void Serial_Init(void)
{
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1, ENABLE);
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9; //TX引脚, 发送模式, 输出数据, 一根线可以有一个输出, 但是可以有多个输入, 接蓝牙模块的RX引脚
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10; //RX引脚, 接收模式, 输入数据, 接蓝牙模块的TX引脚
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    USART_InitTypeDef USART_InitStructure;
    USART_InitStructure.USART_BaudRate = 9600; //波特率
    USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None; //硬件流控
    USART_InitStructure.USART_Mode = USART_Mode_Tx | USART_Mode_Rx; //串口模式
    USART_InitStructure.USART_Parity = USART_Parity_No; //校验位
    USART_InitStructure.USART_StopBits = USART_StopBits_1; //停止位1位
    USART_InitStructure.USART_WordLength = USART_WordLength_8b; //字长, 不需要校验, 那字长就选择8位
    USART_Init(USART1, &USART_InitStructure);

    USART_ITConfig(USART1, USART_IT_RXNE, ENABLE); //开启RXNE标志位到NVIC的输出

    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);

    NVIC_InitTypeDef NVIC_InitStructure;
    NVIC_InitStructure.NVIC_IRQChannel = USART1_IRQn; //中断通道
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1; //优先位
    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
    NVIC_Init(&NVIC_InitStructure);

    /*
    蓝牙模块
    */
    if(Serial_GetRxFlag() == 1)
    {
        RxData = Serial_GetRxData();
    }
    if(RxData == 0x40)
    {
        Go_Ahead();
    }
    if(RxData == 0x41)
    {
        Go_Back();
    }
    if(RxData == 0x42)
    {
        Turn_Left();
    }
    if(RxData == 0x43)
    {
        Turn_Right();
    }
    if(RxData == 0x44)
    {
        Car_Stop();
    }
    if(RxData == 0x45)
    {
        Self_Right();
    }
    if(RxData == 0x46)
    {
        Self_Left();
    }
}

```

(6) 超声波避障模块

下面的代码中, 在写的时候已经有仔细的思考后详细地注释了, 这里就不过多

的说明了。

```
#include "stm32f10x.h"           // Device header
#include "Delay.h"
int16_t Num;
void HCSR_Init(void)
{
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_TIM1, ENABLE);
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP; //推挽输出，一个输出，多个输入
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_14; //triger
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOB, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPD; //下拉输入
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_13; //echo
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOB, &GPIO_InitStructure);

    TIM_InternalClockConfig(TIM1); //72MHz

    TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure; //这种写的后面再配一个PWM波就不用写了
    TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1;
    TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseInitStructure.TIM_Period = 60000 - 1; //ARR
    TIM_TimeBaseInitStructure.TIM_Prescaler = 72 - 1; //PSC
    TIM_TimeBaseInitStructure.TIM_RepetitionCounter = 0;
    TIM_TimeBaseInit(TIM2, &TIM_TimeBaseInitStructure);
}

float HCSR_Distance(void)
{
    TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure; //这种写的后面再配一个PWM波就不用写了
    TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1;
    TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseInitStructure.TIM_Period = 60000 - 1; //ARR
    TIM_TimeBaseInitStructure.TIM_Prescaler = 72 - 1; //PSC
    TIM_TimeBaseInitStructure.TIM_RepetitionCounter = 0;
    TIM_TimeBaseInit(TIM2, &TIM_TimeBaseInitStructure);
}

float HCSR_Distance(void)
{
    GPIO_ResetBits(GPIOB, GPIO_Pin_14);
    GPIO_SetBits(GPIOB, GPIO_Pin_14); //triger发出一个高电平
    Delay_us(20);
    GPIO_ResetBits(GPIOB, GPIO_Pin_14);

    while(GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_13) == RESET); //如果高电平了，就跳出循环，高电平的时候就开始计时，cnt是系统时钟以72兆赫兹给你计时
    // cnt/1000000，就得出计时了多少秒，1秒系统计1000000次
    TIM1->CNT = 0; //CNT清零，CNT就是计数了多少次
    TIM_Cmd(TIM1, ENABLE); //如果echo15引脚收到高电平，那时钟开启
    while(GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_13) == SET); //如果低电平了，就跳出循环，
    TIM_Cmd(TIM1, DISABLE); //TIM关闭
    Num = TIM_GetCounter(TIM1);
    float Distance = (Num * 1.0 * 0.034) / 2; //时间乘声速为距离
    Delay_ms(100); //每次重新工作就停一下
    return Distance; //cnt除以10的6次方，因为系统72MHz，预分频为72000000/72=10的6次方，即1秒测10的6次方，
    //我们共测了CNT次，则cnt/1000000*0.034*1000000，因为340是秒要化为微秒，1秒为1000000微秒
}
```

(7) Main 函数

首先要包含所有头文件，然后在 int main 函数中初始化所有模块，这里代码

展示的是红外巡检，蓝牙控制模块和避障模块在上面代码截图中

```
#include "stm32f10x.h"           // Device header
#include "Delay.h"
#include "OLED.h"
#include "Motor.h"
#include "PWM.h"
#include "Car.h"
#include "Servo.h"
#include "Serial.h"
#include "HCSR.h"
#include "Track.h"
uint8_t RxData;
int main(void)
{
    CAR_Init();
    Serial_Init();
    Servo_Init();
    Track_Init();
    while(1)
    {
        if(GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_8)==0&&
           GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_6)==0&&
           GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_7)==0&&
           GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_15)==0)
        {
            Go_Ahead();
        }
        else if(GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_8)==1&&
                 GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_6)==1&&
                 GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_7)==1&&
                 GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_15)==1)
        {
            Car_Stop();
        }
        else if(GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_8)==0&&
                 GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_6)==0&&
                 GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_7)==1&&
                 GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_15)==1)
        {
            Car_Stop();
        }
        else if(GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_8)==0&&
                 GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_6)==0&&
                 GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_7)==1&&
                 GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_15)==0)
        {
            Self_Right();
        }
        else if(GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_8)==0&&
                 GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_6)==0&&
                 GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_7)==0&&
                 GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_15)==1)
        {
            Self_Right();
        }
        else if(GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_8)==0&&
                 GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_6)==1&&
                 GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_7)==0&&
                 GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_15)==0)
        {
            Turn_Left();
        }
        else if(GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_8)==1&&
                 GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_6)==1&&
                 GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_7)==0&&
                 GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_15)==0)
        {
            Self_Left();
        }
        else if(GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_8)==1&&
                 GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_6)==0&&
                 GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_7)==0&&
                 GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_15)==0)
        {
            Self_Left();
        }
    }
}
```

3 问题与解决方案

- (1) 首先是接线问题，一开始接线电机反转导致小车自动往后退，通过正确连接好杜邦线后成功解决。
- (2) 其次是硬件固定，因为小车的车架上面没有可以固定红外寻迹模块的钩子之类的，所以我用了热熔胶将芯片粘在车架前方，在寻黑线的时候起到了很好的巡迹作用。
- (3) 另一个问题是供电问题，一开始用的是小电池，发现供电不足，后面换成了充电宝，充电宝又因为自身重量会导致小车前进更慢，所以最后面用了 18650 锂电池组，供电充足并且成功提速

4 局限性分析

- (1) 感知能力缺乏环境建模（如 SLAM），仅支持简单路径规划
- (2) 通信距离在蓝牙有效控制范围 $\leq 15\text{m}$ ，适用场景还是十分受限的
- (3) 扩展接口，未充分利用 STM32 的 CAN/USB 外设，多设备协同能力弱
- (4) 无法让三个模式的代码同时写在 `main` 函数中并且设置优先级去执行功能。只能想实现什么功能的时候才将代码写到 `main` 函数中
- (5) 因为时间问题，没办法将电路画成 PCB 板，只能用杜邦线连接，但是杜邦线的接触性很不好，有时候会线头松动，所以后续有时间升级的话，应该画成个 PCB 板

5 总结与展望

通过这次课程设计，无疑大大提高了我的动手能力和对单片机更进一步的深刻了解。一开始学 STM32，感觉非常的迷惑，不知道这个东西到底是什么，但是通过动手操作后，慢慢的就领悟到了其中的妙处。讲实在，我无法想象自

己能完成这一个小项目，因为我认为一开始我是不会的，能力不足。但是因为我勇于踏出第一步，一开始一点也不懂，慢慢找视频找教程学习，慢慢了解STM32，慢慢学会运用，到最后可以写出自己想要的程序。这个经历对我来说特别宝贵。这辆小车是我自己在寒假做成的，并不是这两周做的，但是因为有这次经历，我才会不断的挑战自己，在大二下学期勇于参加自己曾经认为不可能成功的比赛，勇于参加实验室去完成一些对我来说难度极大的项目。虽然很累，但我确确实实学到了东西。所以这不仅仅是单片机的一个课设，更是激发了我在工科这条路上勇于争取奋斗的激情。

心得体会

(全文用小四号宋体单栏 2 倍行距排版)

指导教师批阅意见：			
成绩评定：			
报告格式 (20 分)	报告内容 (70 分)	心得体会 (10 分)	总分
教师签字：			
年 月 日			

注：成绩评定内容可根据实际情况进行调整。